

M4 Student Notes — ELT and Dataform

Notes for the Day 1 (Mon Oct 5) afternoon module, 2:45–3:10 PM CT: ELT in BigQuery, scheduled queries, and the introduction to Dataform — the tool your migration names for Silver→Gold. Followed by the [demo repo tour](#) (3:10–3:25) and the [Dataform anchor lab](#) (3:25–4:05).

ELT vs ETL, in one paragraph

You already know the trade-off; the only shift in habit is where the transform runs. In ELT you land raw data first and transform where the data lives: BigQuery itself is the transformation engine. Your "transform step" stops being an SSIS data flow moving rows through a server and becomes SQL — plain queries, scripting, or Dataform models — executing inside the warehouse. Load Bronze, then let SQL do Silver and Gold.

Key takeaways

- **BigQuery speaks GoogleSQL** (a standard SQL dialect) with scripting, stored procedures, and UDFs. Your T-SQL instincts transfer — variables, BEGIN...END, IF, WHILE all exist. Watch dialect differences (date functions, CREATE TABLE AS instead of SELECT INTO) rather than learning new concepts.
- **Scheduled queries are the SQL Agent instinct, translated.** A saved query on a daily schedule writing into a destination table — the day's closing demo shows it in one click.
- **Dataform manages transformation as code.** Each model is a .sqlx file: a small config block (what to build — table, view, incremental) plus the SELECT that builds it.
- **ref() replaces hard-coded names.** Reference another model as \${ref("name")} and Dataform derives the dependency graph — no precedence constraints, no manually sequenced package steps.
- **The compiled graph is the pipeline.** Dataform compiles your sqlx into a dependency graph, executes it in order, and logs every action. That graph — reviewable in a pull request — is your Silver→Gold pipeline as code.
- **Declarations bring Bronze into the graph.** Tables landed by something else (Datastream, batch loads) are declared as externally managed sources so staging models can ref() them.
- **Assertions are data-quality tests inside the pipeline** — uniqueness, non-null, custom row conditions. Think constraints with better ergonomics: they run with the pipeline, fail the run, and show up in execution logs. Your M&O team's "how do we know the load is good" question gets answered where the data is made, not in a morning email.
- **The git model is the one you already run.** Development workspace ≈ feature branch; commit/push = code review; release configuration ≈ your deployment; scheduled invocation ≈ the SQL Agent schedule. Your CI/CD discipline for SQL Server deployments applies unchanged.

- **Your architecture doc names Dataform for Silver→Gold.** Of everything today, this module plus the lab is the part you will use at work first.

Your SQL Server world → GCP world

In your current stack	In this module's GCP world
SSIS data flow (transform logic)	Dataform .sqlx models (SQL + config block)
SQL Agent schedule	Dataform release config + Cloud Scheduler — or a one-statement scheduled query
T-SQL stored procedures	BigQuery scripting / stored procedures (GoogleSQL), or Dataform models
Manual data validation after loads	Dataform assertions (run in-pipeline, fail loudly)
SQL Agent proxy / run-as account	Service account the execution runs as
Instance → database → schema.table	Project → dataset → table

Dig deeper

- [Dataform overview](#)
- [Test data quality with assertions](#)
- [Release configurations and scheduling](#)
- [GoogleSQL procedural language \(scripting\)](#)
- [Scheduled queries in BigQuery](#)

Part of the Iowa DOM DoIT Data Engineering student pack — see ../00_schedule.md.